
AI-Driven API Testing Platform

Executive Summary & Architecture Guide

Version: 1.0
Date: May 2026
Status: Draft for Review

Prepared by the Engineering Team

Contents

1	What Is This Platform?	2
1.1	The Problem We Solve	2
1.2	Core Assumptions	2
2	High-Level Architecture (HLD)	3
2.1	The Big Picture	3
2.2	Layer-by-Layer Explanation	4
3	How Data Flows Step by Step	5
3.1	Step 1 — Import Your API Spec	5
3.2	Step 2 — Log In as Each Role	5
3.3	Step 3 — Run Tests in Parallel	5
3.4	Step 4 — AI Analysis & Report	6
4	Component Details	7
4.1	Frontend (React / TypeScript)	7
4.2	API Gateway	7
4.3	Auth Service	7
4.4	Spec Import Service	7
4.5	Test Orchestration Service	7
4.6	Job Queue (Redis + BullMQ)	8
4.7	Workers (Test Executors)	8
4.8	AI Analysis Engine	8
4.9	Report Service	9
5	Data Model	10
6	Security Design	11
7	Deployment & Scaling	12
7.1	Container-First Design	12
7.2	Scaling at Each Level	12
7.3	LLM Hosting	12
8	Technology Stack	13
9	Design Alternatives Considered	14
9.1	LLM Strategy	14
9.2	Queue Technology	14
10	Risks & How We Handle Them	15
11	Cost Estimates	16
12	Implementation Checklist	17
13	Product Roadmap	18

What Is This Platform?

In Plain English

This platform automatically tests your APIs. You give it an OpenAPI/Swagger file, it logs in as different users (Admin, Client, Company), runs all the API calls, checks for problems, and then an AI reviews everything and writes a report for you.

The Problem We Solve

Testing APIs manually is slow, error-prone, and hard to repeat. When you have dozens of endpoints each supporting multiple user roles, it becomes almost impossible to cover every combination by hand.

This platform solves that by:

- **Importing** your OpenAPI spec automatically — no manual setup
- **Logging in** as each role and running the right tests for each
- **Running tests in parallel** so large APIs finish quickly
- **Using AI** to find gaps, anomalies, and missing coverage
- **Generating a report** with results, errors, and AI suggestions

Core Assumptions

Before You Start, The Platform Assumes...

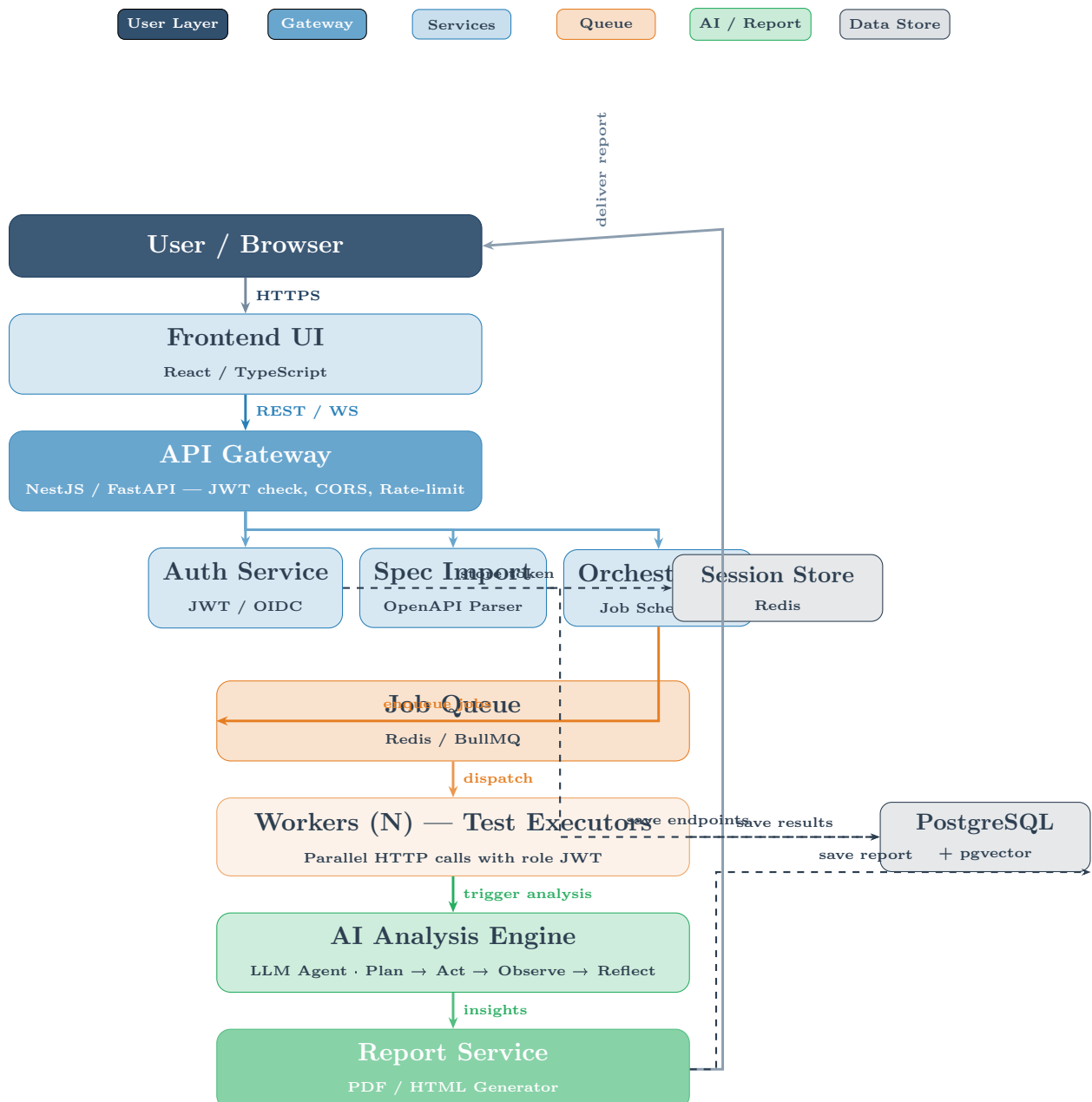
- Your API has an OpenAPI / Swagger specification available at a URL
- You can provide login credentials for each role (Admin, Client, Company, etc.)
- Your API uses JWT, OAuth, or session-based authentication
- You have access to at least one LLM (local via Ollama or cloud via API)
- You want test results monitored and reported automatically

High-Level Architecture (HLD)

The Big Picture

The platform is built using a **microservices architecture** — meaning it is split into small, focused services that each do one job well. They communicate through a central gateway and a job queue.

Below is the full flow from user to report:

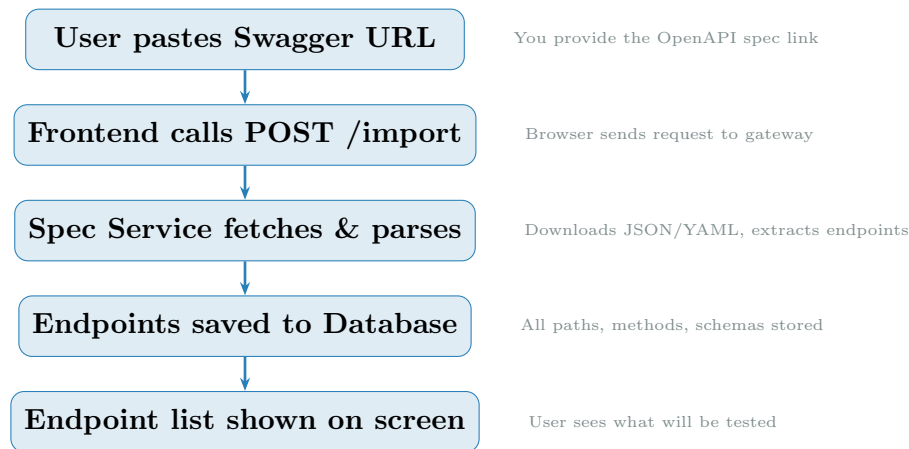


Layer-by-Layer Explanation

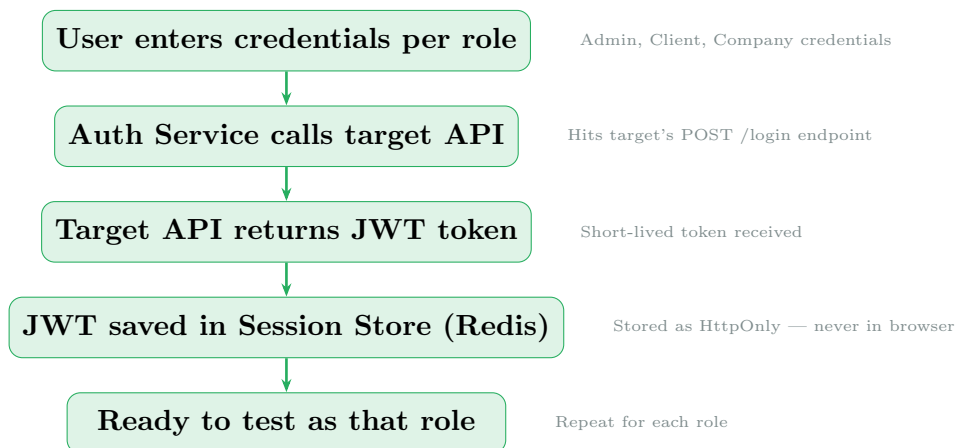
Layer	What It Does
User / Browser	The person who sets up the project, enters credentials, and views the final report.
Frontend UI	A React web app. You click buttons here to import specs, trigger tests, and see results.
API Gateway	The single front door. Every request passes through here. It checks JWTs and routes to the right service.
Auth Service	Logs in as each role on the target API, gets a JWT token, and stores it safely.
Spec Import	Downloads your OpenAPI file, reads all the endpoint definitions, and saves them to the database.
Orchestrator	The coordinator. It decides which tests to run, in what order, and puts them into the job queue.
Job Queue	A list of pending test tasks (powered by Redis/BullMQ). Workers pick from here.
Workers	The actual testers. Each worker picks a job, makes the HTTP call to your API, and records the result.
AI Engine	Reads all results and uses an LLM to spot problems, missing tests, or unusual responses.
Report Service	Combines results + AI insights into a readable PDF or HTML report.
Database	Stores everything: projects, endpoints, sessions, test results, and reports (PostgreSQL + pgvector).

How Data Flows Step by Step

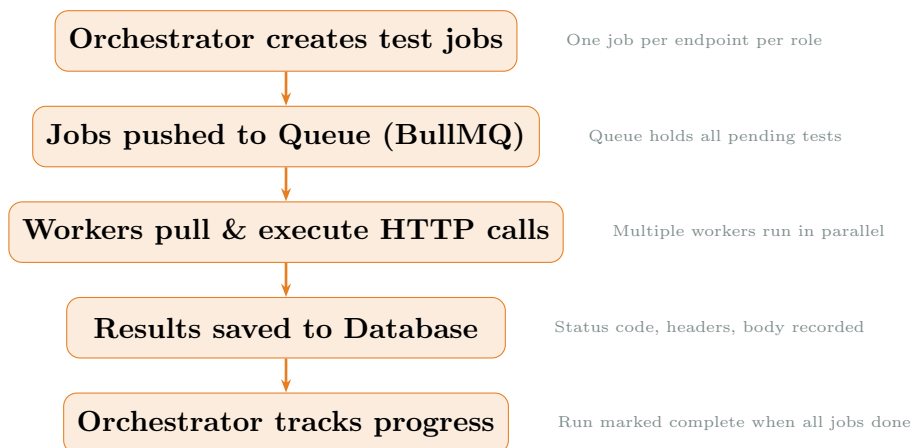
Step 1 — Import Your API Spec



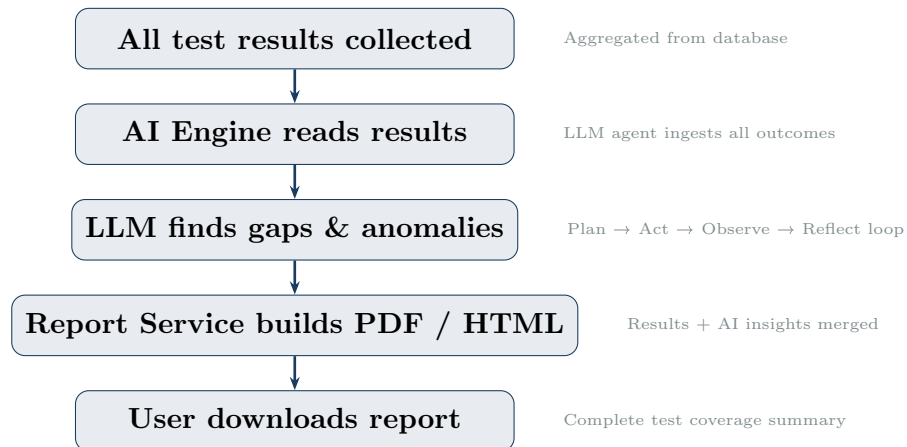
Step 2 — Log In as Each Role



Step 3 — Run Tests in Parallel



Step 4 — AI Analysis & Report



Component Details

Frontend (React / TypeScript)

A web application where you:

- Create or import a project using your Swagger URL
- View all discovered endpoints
- Enter login credentials for each role
- Launch and monitor test runs
- Download the final report

It communicates with the backend exclusively over HTTPS.

API Gateway

Think of this as the reception desk. Every request enters here first. It checks whether you are authenticated, then forwards your request to the correct internal service. It also handles rate-limiting, CORS, and security policies.

Auth Service

When you submit login credentials for a role, this service:

1. Calls the target API's login endpoint
2. Receives a JWT access token back
3. Stores it securely (in Redis / Session Store)
4. Automatically refreshes it before it expires

Tokens are kept in HttpOnly cookies — never in browser localStorage — following OWASP recommendations.

Spec Import Service

Downloads your OpenAPI/Swagger file, then:

- Parses every endpoint, method, and schema
- Extracts parameter definitions and response schemas
- Saves everything to the database so the Orchestrator can use it

Test Orchestration Service

The conductor of the orchestra. It:

- Determines which endpoints need to be tested and in what order
- Handles *dependency chains* — e.g., first POST to create a resource, then GET its ID

- Creates job packets and pushes them to the queue
- Watches progress and marks the run as complete when all jobs finish

Job Queue (Redis + BullMQ)

A first-in, first-out task list. Workers consume from it in parallel. Advantages:

- Jobs are retried automatically if a worker crashes
- Queue depth is monitored — more workers spin up when the backlog grows
- BullMQ is chosen for Node.js environments; RabbitMQ is the alternative for multi-language setups

Workers (Test Executors)

Stateless processes that each:

1. Pull one job from the queue
2. Attach the correct JWT from the session store
3. Make the HTTP call to your API (GET, POST, PUT, DELETE, etc.)
4. Record the status code, headers, and response body to the database
5. Handle chained calls — store a created ID and use it in follow-up requests

Multiple workers run in parallel for speed.

AI Analysis Engine

Once all tests finish, this service:

- Reads all raw test results from the database
- Sends them to an LLM (local or cloud) with a structured prompt
- The LLM follows a **Plan** → **Act** → **Observe** → **Reflect** loop
- Produces structured suggestions: missing test coverage, anomalous responses, security concerns
- All LLM output is validated against a JSON schema before being trusted

Supported LLM backends:

Type	Models	When to Use
Local (Ollama)	DeepSeek-R1, Llama 3, Qwen	Default; no API costs, data stays on-prem
Cloud (API)	Claude, GPT-4, Gemini	Fallback; stronger models, pay-per-use

Report Service

Merges test results and AI insights into a human-readable report (PDF or HTML), including:

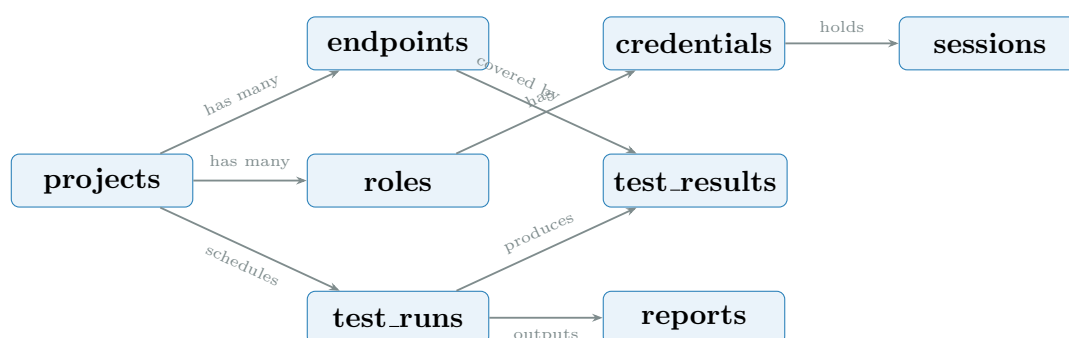
- Pass / fail summary per endpoint and per role
- Error details and response diffs
- AI recommendations highlighted at the top
- Optional: email or Slack notification on completion

Data Model

How Data Is Stored

Everything is stored in a **PostgreSQL** database. The **pgvector** extension is added so the AI engine can store and search vector embeddings alongside regular data — meaning you don't need a separate vector database.

Table	Key Fields	Purpose
projects	id, name, swagger_url	One entry per API you are testing
roles	id, project_id, name	Roles like Admin, Client, Company
credentials	id, role_id, username, password (encrypted)	Login credentials per role
sessions	id, credential_id, access_token, expires_at	Live JWT tokens per role
endpoints	id, project_id, method, path, schemas	Endpoints imported from OpenAPI spec
test_runs	id, project_id, start_time, status	One record per test execution
test_results	id, run_id, endpoint_id, status_code, response	Result of each individual API call
reports	id, run_id, file_url, summary_text	Final report file and AI summary



Security Design

Security Is Built In From The Start

Security is not an afterthought. Every layer of the platform follows OWASP best practices and the principle of **least privilege**.

Area	What We Do
JWT Tokens	Short-lived tokens (under 30 min), auto-refreshed. Stored in HttpOnly cookies — never in localStorage. Signed with RS256 keys.
Role-Based Access	Every endpoint is role-protected. Admins can create/delete; Clients can only read or modify their own data.
Input Validation	All user inputs and API responses are validated against schemas before processing.
Data Encryption	Credentials and tokens are encrypted at rest. All service-to-service traffic uses TLS 1.2+.
AI Output Safety	LLM responses are validated against a JSON schema before being trusted or shown to users. This prevents hallucinations from affecting reports.
Infrastructure	Services run with least-privilege roles. Network policies restrict east-west traffic. WAF sits at the gateway.
Observability	All calls emit metrics and traces (Prometheus + Jaeger). Alerts fire on unusual patterns (spike in 5xx errors, queue backlogs).

Deployment & Scaling

Container-First Design

Every service runs in its own Docker container. Kubernetes (K8s) manages them, providing:

- **Auto-scaling:** More workers spin up automatically as the queue grows (via KEDA)
- **Health checks:** Kubernetes restarts any crashed container automatically
- **Rolling updates:** Deploy new versions with zero downtime

For smaller setups, Docker Compose is supported as a simpler alternative.

Scaling at Each Level

Layer	Scale Trigger	How It Scales
API Gateway	HTTP traffic spikes	Add more gateway pods (HPA)
Workers	Queue depth grows	Auto-scale worker pods (KEDA on Redis)
AI Engine (Local)	Inference demand	Multiple Ollama containers + load balancer
AI Engine (Cloud)	Fallback	Parallel API calls to Claude/GPT-4
Database	Read-heavy load	PostgreSQL read replicas
Queue (Redis)	Throughput limit	Redis Cluster mode

LLM Hosting

Local LLM Setup (Preferred)

Run models like DeepSeek-R1, Llama 3, or Qwen on your own GPU-enabled servers using **Ollama**. Benefits: no per-token API costs, data never leaves your network. Requires: at least one NVIDIA GPU (A100, V100, or 2x RTX 4090).

Technology Stack

Component	Technology	Why This Choice
Frontend	React + TypeScript	Type-safe, fast SPA, large ecosystem
API Gateway	NestJS or FastAPI	NestJS for REST; FastAPI for async/AI workloads
Auth	JWT (RS256) + OIDC	Stateless, OWASP-compliant, supports OAuth providers
Job Queue	Redis + BullMQ	Native Node.js support, TypeScript-friendly, fast
Database	PostgreSQL + pgvector	ACID compliant, vector support avoids a separate vector DB
AI Orchestration	LangChain (Python/Node)	Connects prompts, memory, and tools in one framework
Local LLM	Ollama (DeepSeek, Llama 3)	Free to run, data stays on-prem
Cloud LLM	Claude / GPT-4 / Gemini	Fallback for higher accuracy or when GPU is unavailable
Monitoring	Prometheus + Grafana	Industry standard; works with all services
Tracing	Jaeger (Open-Telemetry)	Traces complex multi-service flows end to end
Infra	Kubernetes + Helm	Declarative, reproducible, auto-scaling

Design Alternatives Considered

LLM Strategy

	Single Local LLM (e.g. DeepSeek)	Multi-Model Fallback
Pros	Simple to set up and tune	Higher resilience; falls back if one model fails
Cons	Single point of failure	More complex (API keys, prompt tuning per model)
Our Choice	✓ Default for MVP	✓ Enabled from v1 onward

Queue Technology

	BullMQ (Redis)	RabbitMQ
Pros	Native Node.js / TypeScript support; fast setup	Durable messages; multi-language; rich routing
Cons	Tied to Redis + Node environment	Requires separate Erlang-based broker
Our Choice	✓ Primary (Node.js stack)	Optional for polyglot setups

Risks & How We Handle Them

Risk	What Could Go Wrong	Mitigation
LLM Hallucinations	AI suggests incorrect or made-up test fixes	<i>Validate all LLM output against JSON schema; present as hints, not automated changes</i>
Token Theft (XSS)	JWT stolen via browser script	<i>Use HttpOnly cookies; add Content Security Policy headers</i>
Queue Overload	Flood of test jobs overwhelms Redis	<i>Set max queue size; auto-scale workers; rate-limit job creation</i>
Target API Down	Workers time out on every call	<i>Retry with exponential backoff; mark job failed after N attempts</i>
Bad OpenAPI Spec	Invalid spec crashes the importer	<i>Pre-validate spec in sandbox; give user clear error message</i>
Concurrency Conflicts	Two workers write to the same DB row	<i>Design jobs as independent; use optimistic locking</i>
Vendor Lock-in	Over-reliance on one LLM provider	<i>Abstract all LLM calls behind an interface; support fallback</i>

Cost Estimates

Tier	Setup	Estimated Cost
Low (On-prem)	All open-source. One GPU workstation for LLMs. Small cloud VMs for services.	\$5–10K upfront \$200–500/month ops
Medium	Cloud LLMs for part of analysis (GPT-4). Managed DB and Redis.	\$1,000+/month ops
High (Enterprise)	Multi-region, multiple GPUs, heavy API usage, monitoring contracts.	\$10K+/month

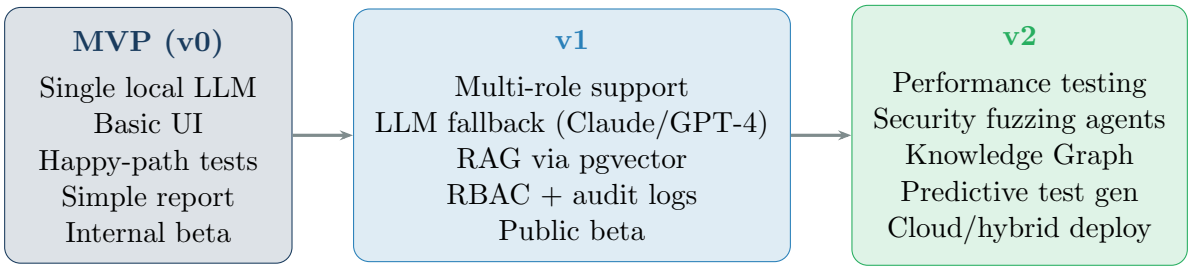
Cost Tip

The biggest cost driver is the LLM strategy. Running DeepSeek or Llama 3 locally on your own GPU hardware eliminates all per-token charges. Cloud models like GPT-4 charge approximately \$0.015–\$0.03 per 1,000 tokens — which adds up fast at scale.

Implementation Checklist

- ☐ **Environments** — Set up Docker/K8s dev and production environments with managed DB and Redis
- ☐ **Database Schema** — Implement all tables and migrations (projects, roles, endpoints, runs, results)
- ☐ **Auth Service** — Build login/logout, integrate with target API auth, generate and store JWTs
- ☐ **Spec Importer** — Implement OpenAPI fetch/parse using swagger-parser or openapi-parser; save endpoints
- ☐ **API Gateway** — Configure routing to each service, JWT middleware, CORS, rate-limiting, monitoring
- ☐ **Orchestrator** — Schedule jobs, resolve endpoint dependency chains, enqueue tasks to BullMQ
- ☐ **Workers** — Read from queue, make HTTP calls with correct session, handle chained requests, write results
- ☐ **Queue Setup** — Deploy BullMQ (or RabbitMQ), configure retries, dead-letter queues, and monitoring
- ☐ **AI Engine** — Integrate LangChain + Ollama; write prompt templates; add output validators; configure fallback
- ☐ **Report Generator** — Build HTML/PDF templates combining results and AI insights (Puppeteer or wkhtmltopdf)
- ☐ **Frontend UI** — Build project setup, endpoint viewer, credential input, results dashboard, and report download
- ☐ **Logging & Monitoring** — Instrument all services with Prometheus metrics; set up Grafana + Jaeger
- ☐ **Security Measures** — Add TLS/HTTPS, CSP headers, input validation; run SAST/DAST scans
- ☐ **Testing** — Unit tests per service, end-to-end tests (Playwright); target 90%+ coverage before release
- ☐ **Documentation** — Swagger UI for platform APIs; user guide; CI/CD pipeline with versioning

Product Roadmap



Each phase includes regression testing, user feedback collection, and incremental improvements to ease-of-use and observability before the next phase begins.

Summary

This platform replaces manual API testing with an automated, AI-assisted pipeline. You import a spec, provide credentials, and get back a detailed report — including AI-generated insights — without writing a single test by hand. The architecture is built to scale horizontally, run LLMs locally (cost-free), and stay secure by design.